

Language Agent Tree Search

Practice Problems – Answer Key

These practice problems accompany the LATS lecture. Work through the video and the slides before starting. This problem set is intended for advanced undergraduates or first-year graduate students, and the whole set should take **30 minutes to an hour**.

- A calculator is useful for Part 2. Part 3 needs a web browser; Parts 1 and 2 do not.
- The reference sheet on the last page carries all important technical details you may need.
- Round to three decimal places. Small differences from the numbers shown in the visualizer are rounding, not mistakes.
- Questions marked *optional* are there if you want to go further.

Part 1: Concepts

Mark one option per question.

Question A.

In LATS, when is the value $V(s)$ of a node computed?

- ☐ (a) Before the action is taken, from the text of the proposed action alone.
- ☐ (b) After the action has been executed in the environment and the observation has come back. **[correct]**
- ☐ (c) Only at terminal nodes, once the environment has returned a reward.
- ☐ (d) During backpropagation, from the rewards of the node's descendants.

Note. Option (a) is Tree of Thoughts. Scoring a step only after it has actually run is the paper's stated distinction, and it is why the value estimate is an observation rather than an opinion.

Question B.

A node s has $V(s) = 0.40$ and $N(s) = 3$. Its parent p has been visited $N(p) = 8$ times. The exploration weight is $w = 1$. Using $\ln 8 \approx 2.0794$, what is $\text{UCT}(s)$?

- ☐ (a) 0.77
- ☐ (b) 0.83
- ☐ (c) 1.09
- ☐ (d) 1.23 **[correct]**

Answer. $\text{UCT}(s) = V(s) + w\sqrt{\ln N(p)/N(s)} = 0.40 + \sqrt{2.0794/3} = 0.40 + \sqrt{0.6931} = 0.40 + 0.833 = 1.233$. Each distractor is a specific mistake: (a) 0.77 swaps $N(p)$ and $N(s)$, giving $0.40 + \sqrt{1.0986/8}$; (b) 0.83 is the exploration bonus alone, with $V(s)$ never added; (c) 1.09 drops the square root, giving $0.40 + 2.0794/3$.

Note. Because $\ln 8 = 3 \ln 2$, the quantity $2.0794/3$ equals 0.6931 exactly, which is the number a student writes down one step before taking the root. Distractor (c) is therefore the answer of someone who stops one step early. Note also that $\ln 3$ on the reference sheet is needed only to check (a).

Question C.

The LATS paper ablates three components on HotPotQA, where the full system scores 0.63 exact match. Which single change costs the most accuracy?

- ☐ (a) Removing reflection.
- ☐ (b) Replacing Monte Carlo Tree Search with plain depth-first search.
- ☐ (c) Removing the language model's value estimate. **[correct]**
- ☐ (d) All three cost roughly the same amount.

Note. From 0.63, the three ablations fall to 0.58 (reflection), 0.42 (depth-first search) and 0.37 (no value estimate). The value function does most of the work, and reflection, the one operation with no counterpart in classical MCTS, is the smallest of the three.

Question D.

In the `openai_multihop-qa` trace, the agent is asked at which venue, and in which year, the paper that introduced the algorithm LATS adapts was presented. Its first trajectory answers "*European Conference on Machine Learning (ECML), 2006*" and receives reward 0. Why is that answer wrong, but tempting?

- ☐ (a) ECML 2006 is where UCT was introduced. The question asks about Monte Carlo Tree Search itself, which Coulom introduced at Computers and Games in 2006. **[correct]**
- ☐ (b) The venue is right, but the year is wrong.
- ☐ (c) ECML did not hold a conference in 2006.
- ☐ (d) The retrieval tool returned no documents, so the agent guessed.

Note. Two different 2006 papers sit behind modern MCTS: Coulom named the method, and Kocsis and Szepesvári derived the UCT selection rule the same year. The agent retrieved a real, relevant document and drew the wrong one of two adjacent facts from it. Only the environment's reward caught it.

Question E.

Two traces solve the same Game of 24 puzzle. In `openai_game-of-24`, a real language model finishes in one iteration having expanded 16 nodes. In `mock_game-of-24`, the offline policy needs five iterations and 45 nodes. What is the right conclusion?

- ☐ (a) The real model is running a better search algorithm.
- ☐ (b) When the policy already proposes a winning action, the search adds cost and buys nothing. **[correct]**
- ☐ (c) Tree search is never worth its cost on reasoning tasks.
- ☐ (d) The offline policy is implemented incorrectly.

Note. Both traces run the same six operations with the same parameters. Search pays off when the policy is uncertain or wrong, not when it is already right: LATS buys robustness, not intelligence.

Part 2: Tracing one round of selection

In the `openai_multihop-qa` trace, the first iteration expands three children of the root, rolls one trajectory out to a terminal state, receives reward 0, and backpropagates it. The second iteration then begins with selection at the root. At that moment the three children are:

- **Node 1** `search[LATS Language Agent Tree Search adapts algorithm venue year]`
- **Node 2** `search["Language Agent Tree Search" paper]`
- **Node 3** `search[UCT algorithm introduced venue year ECML 2006]`

The root has been visited $N(p) = 2$ times and the exploration weight is $w = 1$. Use $\ln 2 \approx 0.6931$.

Question A. Complete the table. Give three decimal places.

Node	$V(s)$	$N(s)$	Exploration bonus $w \sqrt{\ln N(p)/N(s)}$	UCT(s)
1	0.2833	2	0.589	0.872
2	0.5417	1	0.833	1.374
3	0.3917	1	0.833	1.224

Answer. $\sqrt{0.6931/2} = \sqrt{0.34655} = 0.589$ and $\sqrt{0.6931/1} = 0.833$. Node 1: $0.2833 + 0.589 = 0.872$. Node 2: $0.5417 + 0.833 = 1.374$. Node 3: $0.3917 + 0.833 = 1.224$. The visualizer reports 0.8720, 1.3742 and 1.2242; the small differences are rounding.

Question B. Which node does selection take?

Answer. Node 2, with the largest UCT of 1.374.

Question C. Immediately after evaluation in iteration 1, node 1 had the *highest* value of the three: $V = 0.5667$, against 0.5417 for node 2. In one or two sentences, explain what changed between then and now.

Answer. Backpropagating reward 0 through node 1 did two things at once. It cut the node's value from 0.5667 to 0.2833, and it raised its visit count from 1 to 2, which shrank its exploration bonus from 0.833 to 0.589. Node 2 was never tried, so it kept both its original value and a full exploration bonus, and it now scores higher.

Question D (optional). Suppose backpropagation had lowered node 1's value to 0.2833 but left its visit count at 1. Would selection still take node 2? Show the comparison.

Answer. Yes. $UCT(1) = 0.2833 + 0.833 = 1.116$, still below node 2's 1.374. Either effect on its own would have been enough to move selection off node 1.

Part 3: Reading an ablation from the visualizer

Open the visualizer at <https://ryquo-labs.github.io/LATS-Visualizer/>. Use the trace picker in the top right to load `mock_game-of-24`, then `mock_game-of-24_greedy`. The search configuration is shown as labelled chips in the header. Stepping the timeline to its *last* step shows the final result, including the iteration and node counts.

Question A. Exactly one field in the search configuration differs between the two traces. Name the field and give its value in each trace.

Field: _____

Value in `mock_game-of-24`: _____

Value in `mock_game-of-24_greedy`: _____

Answer. The field is w , the exploration weight in UCT. The plain run has $w = 1.0$; the greedy run has $w = 0.0$. Everything else matches: $n = 5$, $\lambda = 0.5$, $\text{depth} \leq 3$, same iteration budget, same seed.

Question B. Complete the table from the final step of each run.

Trace	Iterations run	Nodes	Solved?
<code>mock_game-of-24</code>	5	45	yes
<code>mock_game-of-24_greedy</code>	12	64	no

Question C. The greedy run expanded *more* nodes than the full search and still failed to solve the puzzle. Explain why, in terms of the two terms of the UCT formula. Two or three sentences.

Answer. With $w = 0$ the exploration term vanishes and UCT reduces to $V(s)$ alone, so nothing ever rewards a child for being untried. The search still moves between branches, but only reactively: it leaves a branch once backpropagated failures have pulled its average below a sibling's, never because a sibling has gone unexplored. Selection therefore settles into re-expanding whatever currently scores best instead of opening new options, and the extra nodes pile up as deeper continuations of ground already covered.

Note. If a student argues the greedy run gets stuck in one subtree, check the visualizer: it enters all five root branches, so that is not what happens. The damage shows up as depth. Both runs produce the same 23 nodes at depth 2, but the greedy run puts 35 of its 64 nodes at the maximum depth against 16 of 45 for the full search, and its selection target sits at depth 2 in eight of twelve iterations while the full search never selects below depth 1.

Question D (*optional*). Load `openai_game-of-24_hard` and then `openai_game-of-24_hard_wide`. Again exactly one search parameter differs. What does this pair show about a limit of tree search that no number of extra iterations can fix?

Answer. The parameter is n , the number of samples per expansion, which goes from 5 to 12. The narrow run searches correctly for sixteen iterations across 110 nodes and never solves the puzzle, because none of the five actions the policy proposed at the root can reach 24. Selection, backpropagation and reflection all work perfectly on a tree that cannot contain the answer. Search can only choose among actions the policy proposes, so widening the policy, not searching harder, is what fixes it.

Reference sheet

The three equations

Selection (UCT)	$\text{UCT}(s) = V(s) + w \sqrt{\frac{\ln N(p)}{N(s)}}$
Value	$V(s) = \lambda \cdot \text{LM}(s) + (1 - \lambda) \cdot \text{SC}(s)$
Backup	$N(s) \leftarrow N(s) + 1, \quad V(s) \leftarrow \frac{V_{\text{old}}(s) \cdot (N(s) - 1) + r}{N(s)}$

Notation

s	A node, identified with the transcript that reaches it.
p	The parent of s .
$V(s)$	The current value estimate of s .
$N(s)$	The number of visits to s . Initialized to 1, not 0.
w	The exploration weight. Larger w favours children with fewer visits.
λ	The mixing weight between the two sources of value.
$\text{LM}(s)$	The model's own score for the transcript at s .
$\text{SC}(s)$	Self-consistency: of the n actions sampled at the parent of s , the fraction matching s .
r	The reward the environment returns at a terminal state.

The six operations

Selection	Walk down from the root, taking the highest-UCT child, until reaching an unexpanded node.
Expansion	Sample n candidate actions and execute each one in the environment.
Evaluation	Score each new node with $V(s)$, after the environment has replied.
Simulation	Descend greedily by value, in the real environment, to a terminal state.
Backpropagation	Send r up the path to the root, updating V and N at every ancestor.
Reflection	On failure, write a note about why, and add it to the context of later expansions.

Defaults used in the paper

- $n = 5$ samples per expansion, $w = 1$, and N initialized to 1.
- $\lambda = 0.5$ for question answering and Game of 24; $\lambda = 0.8$ for programming and web tasks.
- In the programming setting, simulation is skipped, because every node is already a complete program. The reward is the fraction of tests that pass.

Useful values $\ln 2 \approx 0.6931$ $\ln 3 \approx 1.0986$ $\ln 8 \approx 2.0794$

Using the visualizer <https://ryquo-labs.github.io/LATS-Visualizer/>

Source code: <https://github.com/Ryquo-Labs/LATS-Visualizer>

- The *trace picker* in the top right switches between runs. The *header* shows the task, the configuration as labelled chips (n , w , λ , depth), the best reward, and the policy.
- The *timeline* steps through the run one operation at a time. Its last step reports the iterations run, the node count, and whether the task was solved.
- Clicking a node shows its action, the observation the environment returned, and its current V and N .